

Programmation et Algorithmique I

Test du 26 octobre 2011

Consignes :

- répondez à chaque problème **sur des feuilles séparées** et utilisez uniquement les feuilles de réponses qui vous sont données ;
 - indiquez votre nom, prénom et section sur chaque feuille de réponses ;
 - les notes de cours et personnelles **ne sont pas autorisées** ;
 - si ce n'est pas explicitement demandé dans l'énoncé, **vous ne devez pas tester que les préconditions sont respectées**.
-

Problème 1 : Matrice des sommes progressives

Dans ce problème on considère des matrices carrées contenant des nombres entiers. Pour une matrice A de taille $(n \times n)$ donnée, on définit une matrice S , appelée la matrice des *sommes progressives*, de la manière suivante :

- S est une matrice carrée de même taille $n \times n$ que A ;
- chaque élément S_{ij} correspond à la somme de tous les éléments de A qui se trouvent dans la sous-matrice rectangulaire de A allant du coin supérieur gauche de A à l'élément A_{ij} inclus.

Par exemple :

$$A = \begin{pmatrix} 1 & 5 & 3 \\ 4 & 2 & 2 \\ 3 & 7 & 9 \end{pmatrix} \implies S = \begin{pmatrix} 1 & 6 & 9 \\ 5 & 12 & 17 \\ 8 & 22 & 36 \end{pmatrix}.$$

Une matrice $(n \times n)$ sera représentée en Python par une liste (de longueur n) de listes (de longueurs n). Par exemple, la matrice A ci-dessus sera représentée par `[[1, 5, 3], [4, 2, 2], [3, 7, 9]]`.

Questions :

- a) Quelle est la matrice des sommes progressives pour la matrice suivante ?

$$\begin{pmatrix} 3 & 5 & 3 \\ 3 & 1 & 2 \\ 0 & 2 & 2 \end{pmatrix}$$

- b) Donnez le code Python d'une fonction `progressive_sum(m)` qui retourne la matrice des sommes progressives d'une matrice carrée m passée en paramètre.

Remarques :

- (*précondition*) la matrice m a été correctement définie : les longueurs des listes définissant m sont égales ;
- (*postcondition*) la fonction ne modifie pas la matrice m , elle retourne une nouvelle matrice ;
- vous êtes libres de concevoir un algorithme itératif ou récursif ; et de définir au besoin des fonctions supplémentaires.

Problème 2 : Anagrammes

Cette question vise à travailler sur les chaînes de caractères.

Deux mots sont des anagrammes s'ils sont constitués des mêmes lettres en même nombre, sans considérer l'ordre dans lequel ces lettres sont placées dans le mot. Voici quelques exemples d'anagrammes : romain et manoir ; mineral et laminer ; logarithme et algorithme ; devenir et deviner.

Questions :

- a) Veuillez écrire une fonction `anagrammes(mot1, mot2)`.

Cette fonction prend en entrée deux chaînes de caractères `mot1` et `mot2` qui représentent deux mots. Cette fonction doit retourner `True` si les mots sont des anagrammes l'un de l'autre, `False` sinon.

- b) Veuillez écrire une fonction `scrabble(mot, fichier)`.

Cette fonction prend en entrée deux chaînes de caractères `mot` et `fichier` où `mot` représente un mot, et `fichier` représente le nom d'un fichier texte dont chaque ligne contient un seul mot. Cette fonction doit retourner une liste. Cette liste doit contenir tous les mots présents dans le fichier nommé `fichier` et qui sont anagrammes du mot `mot`.

Remarques :

- (précondition) le fichier existe et est bien formé.
- vous êtes libres de concevoir un algorithme itératif ou récursif ; et de définir au besoin des fonctions supplémentaires.

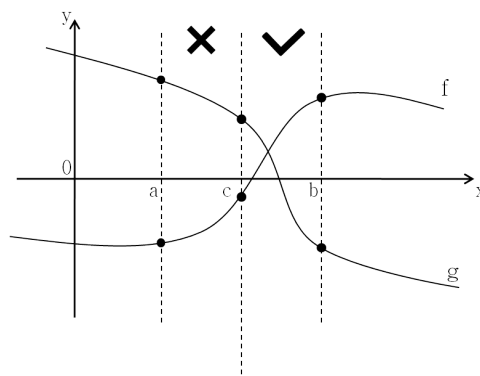
Problème 3 : Intersection de fonctions

Le but de cet exercice est d'approximer l'intersection de deux fonctions continues selon la méthode suivante.

On commence par deux abscisses a et b représentant l'intervalle de recherche. En supposant que l'on cherche l'intersection des fonctions f et g , ces points sont choisis de telle sorte que $f(a) > g(a)$ et $f(b) < g(b)$ (*précondition (*)*). Deux autres préconditions sont que $a < b$ et que f et g sont continues. Notes que ces préconditions impliquent que l'intersection de f et g existe.

A chaque étape de cet algorithme, l'intervalle $[a, b]$ est découpé en deux intervalles de même longueur. Notons-les $[a, c]$ et $[c, b]$. Ensuite,

- si $f(c)$ est "fort proche" de $g(c)$, alors $(c, f(c))$ est une approximation du point d'intersection de f et g ;
- sinon, on passe à l'étape suivante avec l'un des deux intervalles $[a, c]$ ou $[c, b]$ vérifiant la précondition (*).



Question :

- a) Donnez le code python d'une fonction **réursive** `intersection(f, g, a, b, epsilon)` qui étant donnés deux fonctions continues f et g un intervalle $[a, b]$ tel que $f(a) > g(a)$ et $f(b) < g(b)$ et une précision `epsilon`, calcule le point d'intersection de f et g situé dans l'intervalle donné, suivant la méthode décrite précédemment. L'algorithme s'arrête lorsque l'on a trouvé une abscisse c telle que $f(c)$ est `epsilon`-proche de $g(c)$.

Problème 4 : Le codage de Fibonacci

Le codage de Fibonacci est un codage utilisé essentiellement en compression de données. Il utilise les nombres de la suite de Fibonacci, dont les éléments ont la particularité d'être composés de la somme des deux éléments consécutifs précédents.

Le codage de Fibonacci permet d'encoder un nombre entier x en un nombre binaire en suivant la procédure suivante :

1. Créer un tableau avec 2 lignes,
2. Dans la 1ere ligne, mettre les nombres de la suite de Fibonacci à partir de 1 (1, 2, 3, 5, 8, ...),
3. Décomposer l'entier x en une somme d'entiers correspondant aux éléments de la 1ere ligne, en employant les plus grands possibles,
4. Dans la 2e ligne, mettre des 1 en dessous des éléments qui ont permis de décomposer x , sinon mettre 0,
5. Ecrire la 2e ligne et ajouter un 1 à la fin.

Par exemple, l'entier 50 s'encode 001001011 en suivant ce codage :

Suite de Fibonacci	1	2	3	5	8	13	21	34
Présence dans la décomposition	0	0	1	0	0	1	0	1

Il vous est demandé d'implémenter un algorithme qui effectue un tel codage, en vous inspirant de la procédure décrite ci-dessus. Notez bien que, dans le cadre de ce codage, nous considérons que la suite de Fibonacci débute par les éléments 1, 2, 3,

Questions :

- a) Implémentez une fonction **non-réursive** `fibonacci(borne)` qui retourne une liste contenant les nombres de Fibonacci inférieurs ou égaux au paramètre entier `borne` (≥ 1). Ces nombres doivent être en ordre croissant.
- b) Implémentez une fonction **non-réursive** `codage(nombre)` qui retourne une liste d'entiers (de 0 et de 1) correspondant au codage de Fibonacci de l'entier `nombre`.